

Capstone Project — network analysis briefs

Dr. Kanthi Kiran Sirra · datatrends.tech · kk@datatrends.tech

The pipeline is the same for everybody. The *question* is yours. Choose one topic, build it end to end on your own, and submit by the announced date. Four new topics for students who asked for the network analysis domain — telecom call records, a referral graph, firewall logs and router telemetry. These are topics 31 to 34 of the main pack; everything else in that pack applies to them unchanged.

This is not Mini Project 2. Mini Project 2 is the Olist pipeline released in August — it is unchanged, and it is due **Sunday 13 September, 11:59 PM**. The Capstone is separate, comes after that, and is where you choose your own topic. **The Capstone deadline will be announced in the group.**

1 · BRONZE Land the raw file. Change nothing.	2 · SILVER Cast, dedupe, drop orphans. Write down what you decided.	3 · GOLD Aggregate to the answers. Small enough to travel.
4 · EXPORT Gold to one CSV in your Volume.	5 · STAGE Upload to a Snowflake stage.	6 · TABLE COPY INTO, then answer in SQL.

Six things that are true for every topic

Follow the generator exactly.

Every brief gives a seed and exact row counts. Use them without changing them. Your counts must match the ones printed on your page — that is how your work is checked, and it is why the seed is not optional.

Bronze does not clean.

Land every column as a string, add your provenance columns, and stop. Every judgement call belongs in Silver, and every judgement call must be written down.

Use `try_cast`, not `cast`.

ANSI mode is on in Free Edition. `cast` throws and kills the cell; `try_cast` gives you a NULL you can count. Counting those NULLs is the point of the exercise.

Namespace everything on Databricks.

You share one workspace, so your schema, your Volume and your table names all carry your ID. Snowflake is your own account — no prefix is needed there.

Run the load twice.

The second COPY INTO of the same file must report zero files. If it loads the rows again, you

Schedule it.

Wrap your notebooks in a Databricks Job with a schedule and let it run once on its own. A pipeline you have to start by hand is not a pipeline.

have not understood load metadata, and that is worth marks.

Two conventions the code on every page assumes

VOL is your Volume path. Every code snippet writes to a variable called **VOL**. Define it once, at the top of your first notebook, and never type the path again:

```
MY_ID = "yourname"
VOL =
f"/Volumes/workspace/capstone_{MY_ID}/raw"
```

Create the catalog, schema and Volume first, then create the folders under it that your page names.

Sometimes two tables travel, not one.

Most pages give you a Snowflake query that reads your **GOLD_** table. A few questions need row-level detail that Gold has already summarised away, so their query reads a **SILVER_** table instead. When your page does that, send that Silver table to Snowflake by the same route — one CSV, one stage, one COPY INTO. Two tables travel instead of one. That is normal. Gold is not always a single table.

Where a public dataset is named on your page, check its header row before you write a line of code — public files get re-uploaded by many people and the column names drift. Where no reliable public file exists, the page says so plainly and the generator is the only path. That is not a shortcut: a generated dataset with a published seed is more reproducible than a download.

The four network topics

NETWORKS & TELECOM			
31	Cell Tower Health & Dropped Calls	COMFORTABLE	p4
32	The Referral Graph — Who Connects Whom	STRETCH	p8
33	Port Scan Detection in Firewall Logs	STRETCH	p12
34	Interface Bandwidth & the Counter That Wraps	COMFORTABLE	p16

Starter · Comfortable · Stretch describe how much of the work is judgement rather than typing. All three are marked the same way. Choose the theme you would actually enjoy for a week, not the one that sounds hardest.

THE SITUATION

The circle report says 1.82 percent of calls dropped last fortnight and nobody argues with it. Underneath, 40 of the 120 towers fail about one call in eight. They hide because together they carry only 7.69 percent of the traffic, so the healthy towers drown them in the average. The people living under those 40 towers are the ones who change operator. A second trap sits on top: a drop rate is a ratio of sums. Average the per-hour rates instead and the same fortnight reads 4.04 percent.

WHERE THE DATA COMES FROM

There is no public call detail record file. Real CDRs identify people, so operators and the regulator publish aggregates only, never per-call rows with tower_id, start time and end cause. Anything offered as a real CDR dump is either synthetic already or should not be on your laptop. The generator below is the only path, and every number quoted in this brief comes back out of it exactly, on any cluster, in any order.

GENERATE IT – THE EXACT SPECIFICATION

No random draw; every value follows from row position, so it reproduces exactly. towers 120 rows: tower_id T001-T120, site_name, city (6 cities by n%6), district (D1-D4 by n%4), capacity_channels 10 for T001-T080 and 2 for T081-T120, commissioned_on. calls: 120 towers x 15 dates from 2025-06-01 x 24 hours x cnt calls, cnt = reps x w. reps is 6 for T001-T080 and 1 for T081-T120. w is 3 in the busy hours 9-12 and 17-20, 2 in 6-8, 13-16 and 21, 1 elsewhere. That is 4,320 calls per healthy tower and 720 per weak tower, 374,400 in all. Within an hour the cnt calls start evenly, 3600/cnt seconds apart. duration_seconds is 45+(k%4)*15 in quiet hours, 240+(k%4)*120 in medium hours, 420+(k%4)*120 in busy hours. end_cause is DROPPED on every md-th row of a tower-and-w block: md is 360, 180, 72 for healthy towers at w of 1, 2, 3 and 20, 10, 6 for weak towers, giving 40 drops per healthy tower and exactly 90 per weak tower. Then, over rows numbered by call_id, inject on disjoint classes: 1,200 duration_seconds set to 'NA', 900 end_cause padded with spaces and lower-cased, 600 tower_id overwritten to 'T999', 450 duration_seconds set to -1, and 1,800 whole rows repeated, for a file of 376,200.

GENERATOR CODE – RUN THIS FIRST, EXACTLY AS WRITTEN

This assumes MY_ID and VOL are already set, as on the second page of this document. Do not change the seed or any of the counts — your checks later depend on them.

```
import pyspark.sql.functions as F; from pyspark.sql import Window
CI = "element_at(array('Pune','Kochi','Indore','Jaipur','Nagpur','Surat'),cast(id%6 as int)+1)"
PK, MD = '(9,10,11,12,17,18,19,20)', '(6,7,8,13,14,15,16,21)'
t = spark.range(120).selectExpr('id n', "format_string('T%03d',id+1) tower_id",
    f"concat({CI},' Site ',format_string('%03d',id+1)) site_name", f'{CI} city',
    "format_string('D%d',cast(id%4 as int)+1) district", 'if(id<80,10,2) capacity_channels',
    "date_add(date'2019-01-01',cast(id*11 as int)) commissioned_on")
t.drop('n').write.mode('overwrite').option('header',True).csv(f'{VOL}/raw/towers')
c = (t.withColumn('call_date', F.explode(F.expr(
    "sequence(date'2025-06-01',date'2025-06-15')"))))
    .withColumn('hour_of_day', F.explode(F.expr('sequence(0,23)'))))
    .selectExpr('n', 'tower_id', 'call_date', 'hour_of_day', 'if(n<80,6,1) reps',
        f'case when hour_of_day in {PK} then 3 when hour_of_day in {MD} then 2'
        ' else 1 end w')).selectExpr('*', 'w*reps cnt',
    'if(n<80,element_at(array(360,180,72),w),element_at(array(20,10,6),w)) md')
c = (c.withColumn('k', F.explode(F.expr('sequence(1,cnt)'))).selectExpr('*',
    'hour_of_day*3600+(k-1)*cast(3600/cnt as int) ss',
    'element_at(array(45,240,420),w)+(k%4)*element_at(array(15,120,120),w) dur',
    "concat_ws('-',tower_id,string(call_date),string(hour_of_day),string(k)) call_id")
    .withColumn('rb', F.row_number().over(Window.partitionBy('tower_id','w')
        .orderBy('call_date','hour_of_day','k'))))
c = c.selectExpr('*', "case when rb%md=0 then 'DROPPED' when rb%17=1 then 'BUSY'"
    " when rb%23=2 then 'NO_ANSWER' else 'NORMAL' end cause").withColumn(
    'rn', F.row_number().over(Window.orderBy('call_id')))
raw = c.selectExpr('rn', 'call_id',
    "format_string('SUB%05d',cast(pmod(k*37+hour_of_day*11+n*7,60000) as int)) subscriber_id",
    "if(rn%624=2,'T999',tower_id) tower_id",
    "cast(timestampadd(second,ss,cast(call_date as timestamp)) as string) start_ts",
    "case when rn%312=0 then 'NA' when rn%832=3 then '-1' else string(dur) end duration_seconds",
    "if(rn%416=1,concat(' ',lower(cause),' '),cause) end_cause",
    "if(k%2=0,'MT','MO') direction", "if(n%3=0,'5G','4G') technology")
(raw.unionByName(raw.filter('rn%208=5')).drop('rn').write.mode('overwrite')
    .option('header',True).option('ignoreLeadingWhiteSpace',False)
    .option('ignoreTrailingWhiteSpace',False).csv(f'{VOL}/raw/calls'))
```

BRONZE – LAND IT, CHANGE NOTHING

Land towers and calls exactly as written, every column a string. calls must read back as 376,200 rows and towers as 120. Add `_source_file` from `input_file_name()`, `_ingested_at` from `current_timestamp()` and `_row_hash` over the concatenated raw columns. Do not trim `end_cause`, do not cast `duration_seconds`, do not drop the 'T999' rows. How many broken rows the collector sent is a fact about the collector, lost the moment you clean here.

SILVER – THE DECISIONS YOU MUST MAKE, AND RECORD

Each decision below has a number attached to it. If your number does not match, stop and find out why before you go on — that gap is the finding, not a nuisance.

1. CAST with `try_cast`, never cast: `duration_seconds` to INT, `start_ts` to TIMESTAMP. ANSI mode is on, so cast throws on the first bad value and the notebook stops. Exactly 1,200 rows hold the string 'NA' in `duration_seconds` and go NULL. Send those 1,200 to `silver_rejects` with reason 'duration_not_numeric'. A call with no duration is unknown, not a zero-second call, so it leaves both numerator and denominator.
2. DEDUPE on `call_id`. The collector re-sent one slice of its file. 376,200 rows in, 374,400 out, exactly 1,800 removed: ROW_NUMBER over `call_id` ordered by `_ingested_at`, keep rank 1. Check both counts and the difference before going on. Removing more than 1,800 means your key is wrong; removing none means you deduped on something already unique, such as the whole row.
3. ORPHANS and RANGE. Exactly 600 rows carry `tower_id` 'T999', which matches no tower master row, so they can be blamed on nobody: reject, reason 'unknown_tower'. Exactly 450 rows carry `duration_seconds` -1. Reject those with a rule - duration outside 0 to 7200 seconds - and reason 'duration_out_of_range', not by hunting for -1, because the next collector bug sends a different wrong number. Silver = 372,150 rows.
4. END CAUSE, defined once. Exactly 900 rows hold the cause padded with spaces and in lower case, so a raw GROUP BY shows eight causes instead of four and the drop count comes up short. trim then upper before anything else, then set `is_dropped = (end_cause = 'DROPPED')` here and nowhere else. Derive `end_ts = start_ts + duration_seconds` here too; the concurrency work needs it.

GOLD – THE SHAPE THAT TRAVELS

One row per tower per calendar date per hour of day: `tower_id`, `site_name`, `city`, `district`, `capacity_channels`, `call_date`, `hour_of_day`, `calls`, `dropped_calls`, `drop_rate`, `peak_concurrent`, `utilisation`, `avg_duration_seconds`. $120 \times 15 \times 24 = 43,200$ cells, less 58 whose only call was rejected, so exactly 43,142 rows. `peak_concurrent` is the most calls open at once in that cell: emit +1 at `start_ts` and -1 at `end_ts`, running SUM over time, take the max. `utilisation = peak_concurrent / capacity_channels`.

THE TRAP MOST PEOPLE FALL INTO

Writing `AVG(drop_rate)` over the Gold rows, or a rate per tower-hour averaged up to the tower. That is an average of ratios, and it gives a quiet hour with 6 calls and 1 drop the same vote as a busy hour with 18 calls and none. Here it reads 4.04 percent instead of 1.82, more than double, and it reshuffles the order inside the worst 40. Carry `SUM(dropped_calls)` and `SUM(calls)` as two columns to the final query and divide once, at the end, with NULLIF on the denominator. Same rule per day in question 2.

THE THREE QUESTIONS YOUR GOLD LAYER MUST ANSWER

1. Drop rate per tower per hour of day, and the hour each tower is worst
2. Which towers exceed a 5 percent drop rate on more than 10 of the 15 days?
3. Does drop rate rise with concurrent calls on the same tower, or is it flat?

IN SNOWFLAKE – THE HARD ONE, WORKED

Tables this query needs in Snowflake: **GOLD_TOWER_HOUR**

```
-- Q1. Drop rate is SUM(dropped) / SUM(calls): a ratio of sums.
-- The window SUMs put each tower's own overall rate beside its worst hour.
WITH th AS (
  SELECT tower_id, hour_of_day,
         SUM(calls) AS calls, SUM(dropped_calls) AS drops
  FROM   GOLD_TOWER_HOUR
  GROUP BY tower_id, hour_of_day)
SELECT tower_id, hour_of_day, calls, drops,
       ROUND(100.0 * drops / NULLIF(calls, 0), 2) AS worst_hour_pct,
       ROUND(100.0 * SUM(drops) OVER (PARTITION BY tower_id)
            / NULLIF(SUM(calls) OVER (PARTITION BY tower_id), 0), 2) AS tower_pct
FROM   th
QUALIFY ROW_NUMBER() OVER (PARTITION BY tower_id
                          ORDER BY drops / NULLIF(calls, 0) DESC, hour_of_day) = 1
ORDER BY worst_hour_pct DESC;
```

YOU ARE DONE WHEN

- ☐ Bronze calls = 376,200 rows and Bronze towers = 120. Count the file you wrote, not the DataFrame you think you wrote. Reading back 374,400 means the repeated slice never landed.
- ☐ Silver = 376,200 - 1,800 repeated call_id - 1,200 'NA' durations - 600 'T999' - 450 negative durations = 372,150. Print all five numbers and check the subtraction closes.
- ☐ Gold = 43,142 rows, SUM(calls) = 372,150, SUM(dropped_calls) = 6,792, circle rate 1.82 percent. Ranked by ratio of sums the worst 40 tower_ids are exactly T081 to T120, each near 12.5 percent.
- ☐ peak_concurrent is 4 in busy hours on T001-T080, 2 in medium hours, 1 otherwise, and 1 everywhere on T081-T120. Drop rate by utilisation rises 0.27, 0.56, 1.40, 12.57 percent across utilisation 0.1, 0.2, 0.4, 0.5.

IF YOU FINISH EARLY

Add a 7-day trailing drop rate per tower with ROWS BETWEEN 6 PRECEDING AND CURRENT ROW over the daily sums, and flag the first date each tower crosses 5 percent. Then bucket by raw peak_concurrent instead of utilisation and watch it invert: 4.37, 0.56, 1.40 percent. Explain why before believing either number.

THE SITUATION

A referral scheme has 600 members in six cities. The leaderboard names the top referrer: one member brought in 34 people. Look one step further and that reach stops dead, because those 34 already knew each other - only 34 more members sit at two hops. Five other members have about a dozen contacts each and reach more than fifty at two hops, because each is the only link between two cities. Remove one and the network breaks apart. Nobody has noticed, because no column says 'bridge'. You have to join the edge list to itself to find them.

WHERE THE DATA COMES FROM

There is no public referral-graph file of this shape. Real referral tables belong to one company and identify people, and published network datasets carry none of referrer, referred, date, channel and status together. So the generator below is the only path. One thing to settle before you start: GraphFrames is NOT available on Databricks Free Edition serverless and no third-party library can be installed, so do not go looking for it. Everything here is self-joins on the edge table in plain PySpark.

GENERATE IT — THE EXACT SPECIFICATION

Determined by row position, no random draw. members 600 rows: member_id M001-M600, city (6 cities, M001-M100 in the first), joined_on, segment by id%3. Write m as 0-599, city $c = m/100$, local $l = m\%100$. Edges come in four blocks, numbered in that order. ring: every member refers $(l+s)\%100$ in its own city, s in [1,2,3,5,8,13] when $l\%4=0$, [1,2,3,5] when 1, [1,2,3] when 2, [1,2] when 3 - 2,250 rows. hub: the member at $l=0$ in each city refers 30 more, s = 21 to 50 - 180 rows. mutual: for every $l\%10=0$ the member at $l+1$ refers back - 60 reciprocal pairs. bridge: the five members at $l=50$ in the first five cities each refer 6 members at $l=50$ to 55 in the next city - 30 rows, the only edges that leave a city. Total 2,520. referred_at spreads over 2025-01 to 2025-06 by step, so the network grows month on month. status is JOINED except on ring rows with $s>1$, where every 25th is REJECTED and the next PENDING. Then, over rows numbered by build order, inject on disjoint classes: 45 self-referrals (referred_id set to referrer_id), 60 rows with referred_id 'M999' matching no member, 90 rows with status padded and lower-cased, 72 with referred_at 'NA', and 120 whole rows repeated - a file of 2,640.

GENERATOR CODE – RUN THIS FIRST, EXACTLY AS WRITTEN

This assumes MY_ID and VOL are already set, as on the second page of this document. Do not change the seed or any of the counts — your checks later depend on them.

```
import pyspark.sql.functions as F; from pyspark.sql import Window
CI = "element_at(array('Pune','Kochi','Indore','Jaipur','Nagpur','Surat'),id div 100+1)"
ST = "element_at(array(array(1,2,3,5,8,13),array(1,2,3,5),array(1,2,3),array(1,2)),l%4+1)"
m = spark.range(600).selectExpr("format_string('M%03d',id+1) member_id", f'{CI} city',
    "date_add(date'2024-06-01',cast(id%180 as int)) joined_on",
    "element_at(array('STUDENT','PRO','SMB'),cast(id%3 as int)+1) segment",
    'id mno', 'id div 100 c', 'cast(id%100 as int) l')
m.drop('mno','c','l').write.mode('overwrite').option('header',True).csv(f'{VOL}/raw/members')
ring = m.withColumn('s', F.explode(F.expr(ST))).selectExpr('mno src',
    'c*100+(l+s)%100 dst', 'element_at(map(1,6,2,5,3,4,5,3,8,2,13,1),s) mon', '0 grp', 's sk')
hub = (m.filter('l=0').withColumn('s', F.expr('explode(sequence(21,50))')))
    .selectExpr('mno src', 'c*100+(l+s)%100 dst', '4+s%3 mon', '1 grp', 's sk'))
mut = m.filter('l%10=0').selectExpr('c*100+(l+1)%100 src', 'mno dst', '5 mon',
    '2 grp', '0 sk')
brg = (m.filter('mno in (50,150,250,350,450)').withColumn('j', F.expr('explode(sequence(0,5))')))
    .selectExpr('mno src', '(c+1)*100+50+j dst', '6 mon', '3 grp', 'j sk'))
e = (ring.unionByName(hub).unionByName(mut).unionByName(brg)
    .withColumn('rn', F.row_number().over(Window.orderBy('grp', 'src', 'sk'))))
    .selectExpr('*', "case when grp=0 and sk>1 and rn%25=0 then 'REJECTED'"
    " when grp=0 and sk>1 and rn%25=1 then 'PENDING' else 'JOINED' end st0"))
raw = e.selectExpr('rn', "format_string('R%05d',rn) referral_id",
    "format_string('M%03d',src+1) referrer_id",
    "case when rn%56=0 then format_string('M%03d',src+1) when rn%42=1 then 'M999'"
    " else format_string('M%03d',dst+1) end referred_id",
    "case when rn%35=3 then 'NA' else string(date_add("
    "add_months(date'2025-01-01',mon-1),cast(rn%28 as int))) end referred_at",
    "element_at(array('APP','SMS','EMAIL','EVENT'),cast(rn%4 as int)+1) channel",
    "if(rn%28=2,concat(' ',lower(st0),' '),st0) status")
(raw.unionByName(raw.filter('rn%21=4')).drop('rn')).write.mode('overwrite')
    .option('header',True).option('ignoreLeadingWhiteSpace',False)
    .option('ignoreTrailingWhiteSpace',False).csv(f'{VOL}/raw/referrals'))
```

BRONZE – LAND IT, CHANGE NOTHING

Land members and referrals as written, every column a string. referrals must read back as 2,640 rows and members as 600. Add `_source_file`, `_ingested_at` and `_row_hash`. Say it once now: GraphFrames is not installable on Free Edition and you will not use it. Every question below is answered by joining the edge table to itself - one join for mutual pairs and two-hop, three for triangles - so your job in Silver is an edge table clean enough that a self-join cannot lie to you.

SILVER – THE DECISIONS YOU MUST MAKE, AND RECORD

Each decision below has a number attached to it. If your number does not match, stop and find out why before you go on — that gap is the finding, not a nuisance.

1. CAST `referred_at` to DATE with `try_cast`, never `cast`, because ANSI mode makes cast throw. Exactly 72 rows hold 'NA' and go NULL: reject with reason 'no_referral_date'. Question 1 needs the month, and an edge with no date cannot be placed in one, so it must not be quietly kept as month zero.
2. DEDUPE on `referral_id`. 2,640 rows in, 2,520 out, exactly 120 removed. Use `referral_id`, not the pair (`referrer_id`, `referred_id`): the injected self-referrals and 'M999' rows carry their own `referral_id` and have to survive dedupe so you can count and reject them properly in the next step.
3. SELF-REFERRALS and UNKNOWN MEMBERS. Exactly 45 rows have `referrer_id` = `referred_id` - reject, reason 'self_referral'; a loop is not a link and it shows up later as a fake triangle. Exactly 60 rows have `referred_id` = 'M999', which matches no member - reject, reason 'unknown_member', found by an anti-join against members rather than a hardcoded 'M999'. Silver = 2,520 - 72 - 45 - 60 = 2,343 rows.
4. STATUS once, then the link table. Exactly 90 rows hold status padded with spaces and lower-cased, so trim then upper first. Keep only status = 'JOINED': 2,227 rows, setting aside 87 REJECTED and 29 PENDING, because a pending invite is not a relationship. Then build LINKS, one row per undirected pair: `least(referrer_id, referred_id) AS a_id, greatest(...) AS b_id, DISTINCT` - 2,175 rows.

GOLD – THE SHAPE THAT TRAVELS

Two tables. `gold_member_month`, one row per member per month, dense so zero months are present: `member_id`, `city`, `segment`, `month`, `out_edges`, `in_edges`, `cum_out_degree`, `cum_in_degree` - 600 x 6 = exactly 3,600 rows, with `SUM(out_edges) = SUM(in_edges) = 2,227`. `gold_member_graph`, one row per member: `member_id`, `city`, `segment`, `out_degree`, `in_degree`, `undirected_degree`, `mutual_pairs`, `triangles`, `one_hop`, `two_hop_exact`, `cities_within_two` - exactly 600 rows, none missing, none isolated.

THE TRAP MOST PEOPLE FALL INTO

Counting each undirected pair twice. A referral row is directed, so a self-join returns (A,B) and (B,A), every pair counts double and every triangle six times. Fix it once in Silver: build LINKS with `least()/greatest()` and `DISTINCT`, then keep an ordering predicate `a.x < b.x` in every self-join after. Two more: two-hop without an anti-join against the one-hop set and against self counts direct contacts twice and every member as reaching itself; and keeping PENDING and REJECTED invents 116 links nobody accepted.

THE THREE QUESTIONS YOUR GOLD LAYER MUST ANSWER

1. Out-degree and in-degree per member, and how each changed month by month
2. Mutual pairs and closed triangles - which members sit in the most?
3. Two-hop reach: how many members are reachable in exactly two steps, not one?

IN SNOWFLAKE – THE HARD ONE, WORKED

Tables this query needs in Snowflake: **LINKS**, **GOLD_MEMBER_GRAPH**

```
-- Q2. Triangles by joining the edge list to itself three times.
-- LINKS already holds each undirected pair once with a_id < b_id, so
-- forcing x < y < z counts each triangle once instead of six times.
WITH t AS (
  SELECT e1.a_id AS x, e1.b_id AS y, e2.b_id AS z
  FROM   LINKS e1
  JOIN   LINKS e2 ON e2.a_id = e1.b_id AND e2.b_id > e1.b_id
  JOIN   LINKS e3 ON e3.a_id = e1.a_id AND e3.b_id = e2.b_id),
p AS (SELECT x AS member_id FROM t
      UNION ALL SELECT y FROM t
      UNION ALL SELECT z FROM t)
SELECT p.member_id, g.city, g.undirected_degree AS degree,
       COUNT(*) AS triangles
FROM   p JOIN GOLD_MEMBER_GRAPH g ON g.member_id = p.member_id
GROUP BY p.member_id, g.city, g.undirected_degree
QUALIFY RANK() OVER (ORDER BY COUNT(*) DESC) <= 10
ORDER BY triangles DESC, p.member_id;
```

YOU ARE DONE WHEN

- ☐ Bronze referrals = 2,640 and members = 600. After dedupe on referral_id, 2,520: exactly 120 removed. Count the written file, not the DataFrame.
- ☐ Silver = 2,520 - 72 unreadable dates - 45 self-referrals - 60 unknown members = 2,343. Of those 2,227 are JOINED, 87 REJECTED, 29 PENDING, and $2,227 + 87 + 29 = 2,343$.
- ☐ LINKS = 2,175 undirected pairs and mutual pairs = 52, so $2,175 + 52 = 2,227$ joined edges: each reciprocal pair is two edges folded into one link. Triangles = 2,164, and $\text{SUM}(\text{triangles})$ over the 600 members = $6,492 = 3 \times 2,164$.
- ☐ gold_member_graph = 600 rows, $\text{SUM}(\text{undirected_degree}) = 4,350 = 2 \times 2,175$, and no member has one_hop = 0. Exactly 5 members have two_hop_exact of 52 or more - M051, M151, M251, M351, M451 - while the four largest out-degrees, 34 each, reach only 33 to 36.

IF YOU FINISH EARLY

Snowflake's recursive CTE reaches depth beyond two - three, four, or the whole component - which self-joins cannot do without one join per hop. It is not needed for these three questions, but write it and compare hop 2 with your PySpark answer. Then drop the 30 bridge edges and recount two_hop_exact.

THE SITUATION

Nothing alerted. The firewall did its job all morning and denied what it was told to deny. A scan is not one strange packet - it is one source touching many ports in a short window, and it reads as a busy client until you count distinct destination ports per source per minute. In this three hours, 27.5% of every denied connection comes from three addresses that were never once allowed through. Three sources cross 100 distinct ports inside a five-minute window, and one of the three is your own backup server.

WHERE THE DATA COMES FROM

Raw firewall text of this shape is hard to get honestly. The captures people name - CICIDS2017, UNSW-NB15, the LANL 2017 release - are flow records or labelled feature tables, not firewall log lines; several sit behind a registration form, the files are large, and the column layout changes between releases. Do not quote a schema for one of them here without opening it yourself first. There is no small public file with the line shape this project needs, so the generator below is the primary path and the only one whose numbers are checkable.

GENERATE IT – THE EXACT SPECIFICATION

Seed 33. One text file, 233,200 lines, all 2025-04-08, 180 one-minute buckets from 06:00. Line: 'YYYY-MM-DD HH:MM:SS fw-edge-01 <proto> <ACTION> <src>:<sport> > <dst>:<dport> bytes=<n|-> eid=E<9 digits>'. Body 224,700 events, each with its own eid. Chatter: 200 sources 10.30.0.1-10.30.3.50, 6 events a minute each, 216,000 rows, dst ports from a fixed set of 8 so each shows 6 distinct ports a minute; DENY where $(6s+r) \bmod 25 = 0$, exactly 8,640; sources 10.30.0.31-10.30.0.38 send 1 ICMP event a minute with both ports written '-', exactly 1,440; 10.30.0.5, 10.30.0.9 and 10.30.0.13 write accept in lower case, exactly 2,700, every accept they ever send. Scanners, all DENY, never an accept: 203.0.113.45 hits 10.30.9.7 on ports 1-1200 over minutes 30-39, 120 a minute; 198.51.100.22 hits ports 1000-1899 over minutes 20-109, 10 a minute; 10.30.7.99 hits 20 hosts on ports 1024-2223 over minutes 100-114, 80 a minute. Backup 10.30.8.20 streams to 8 storage hosts on ports 20000-20999, 30 a minute for all 180 minutes: 5,400 rows, 60 DENY, 5,340 ACCEPT. Every DENY writes bytes='-'. Defects: chatter minutes 0-4 repeated, 6,000 lines; 2,000 truncated after the protocol; 500 blank.

GENERATOR CODE – RUN THIS FIRST, EXACTLY AS WRITTEN

This assumes MY_ID and VOL are already set, as on the second page of this document. Do not change the seed or any of the counts — your checks later depend on them.

```
P = "array(22,80,443,3306,53,123,8080,9000)"
a = spark.range(216000).selectExpr("id", "cast(id div 1200 as int) m",
    "cast(pmod(id div 6, 200) as int) s", "cast(pmod(id, 6) as int) r")
a = a.selectExpr("m", "format_string('10.30.%d.%d', s div 50, pmod(s, 50) + 1) src",
    "format_string('10.30.9.%d', pmod(id, 20) + 1) dst",
    f"if(s between 30 and 37 and r = 0, '-', cast(element_at({P}, "
    " pmod(6*s + r, 8) + 1) as string)) dps",
    "if(s between 30 and 37 and r = 0, 'icmp', 'tcp') pr",
    "if(pmod(6*s + r, 25) = 0, 'DENY', if(s in (4,8,12), 'accept', 'ACCEPT')) act",
    "if(pmod(6*s + r, 25) = 0, '-', cast(200 + pmod(id, 9000) as string)) by",
    "format_string('E%09d', id) eid")
sc = spark.range(3300).selectExpr(
    "cast(if(id < 1200, 30 + id div 120, if(id < 2100, 20 + (id - 1200) div 10, "
    " 100 + (id - 2100) div 80)) as int) m",
    "if(id < 1200, '203.0.113.45', if(id < 2100, '198.51.100.22', '10.30.7.99')) src",
    "if(id < 1200, '10.30.9.7', format_string('10.30.50.%d', pmod(id, 20) + 1)) dst",
    "cast(if(id < 1200, 1 + id, if(id < 2100, id - 200, id - 1076)) as string) dps",
    "'tcp' pr", "'DENY' act", "'-' by", "format_string('E%09d', 1000000 + id) eid")
bk = spark.range(5400).selectExpr("cast(id div 30 as int) m", "'10.30.8.20' src",
    "format_string('10.30.50.%d', pmod(id, 8) + 1) dst",
    "cast(20000 + pmod(id, 1000) as string) dps", "'tcp' pr",
    "if(pmod(id, 90) = 0, 'DENY', 'ACCEPT') act",
    "if(pmod(id, 90) = 0, '-', cast(9000 + pmod(id, 900) as string)) by",
    "format_string('E%09d', 2000000 + id) eid")
LN = ("concat_ws(' ', date_format(timestamp'2025-04-08 06:00:00' + make_dt_interval("
    "0, 0, m, cast(pmod(crc32(eid), 60) as decimal(8,6))), 'yyyy-MM-dd HH:mm:ss'),"
    " 'fw-edge-01', pr, act, concat(src, ':', if(pr = 'icmp', '-', "
    " cast(40000 + pmod(crc32(eid), 9000) as string))), '>',"
    " concat(dst, ':', dps), concat('bytes=', by), concat('eid=', eid)) value")
tr = spark.range(2000).selectExpr("concat('2025-04-08 06:', "
    " lpad(cast(pmod(id, 60) as string), 2, '0'), ':00 fw-edge-01 tcp') value")
(a.union(sc).union(bk).selectExpr(LN).union(a.filter('m < 5').selectExpr(LN))
    .union(tr).union(spark.range(500).selectExpr("' ' value"))
    .write.mode('overwrite').text(f'{VOL}/raw/fw_logs'))
```

BRONZE – LAND IT, CHANGE NOTHING

`spark.read.text` gives one column, value, holding the raw line - that is Bronze. Add `_source_file`, `_ingested_at` and `_line_length` and stop. Do not parse, do not drop the blank or truncated lines, do not lower-case anything. Bronze is exactly 233,200 rows and that number must equal the line count of the file. A scan hides in the rows a pipeline threw away before anyone looked, so the count of lines you could not read is a published number, not a silent filter.

SILVER – THE DECISIONS YOU MUST MAKE, AND RECORD

Each decision below has a number attached to it. If your number does not match, stop and find out why before you go on — that gap is the finding, not a nuisance.

1. PARSE with one anchored regex capturing timestamp, device, protocol, action, `src_ip`, `src_port`, `dst_ip`, `dst_port`, bytes and eid. Exactly 2,500 lines yield nothing - 2,000 truncated after the protocol and 500 blank - and every one goes to a reject table with its raw line. Parse rate $230,700 / 233,200 = 98.93\%$. Publish that number.
2. DEDUPE ON eid, which is why the generator writes one. 230,700 parsed rows fall to exactly 224,700, so 6,000 go, and the DENY count falls from 12,240 to 12,000. Print both. Never dedupe on the raw line: two real connections from one host in the same second to the same port are identical text and both of them happened.
3. PORTS AND BYTES ARE NULL, NOT ZERO. Exactly 1,440 ICMP events write '-' for both ports; `try_cast` to INT gives NULL, and `coalesce` to 0 invents a port 0 that adds a fake distinct port to eight sources. Exactly 12,000 DENY rows write `bytes='-'`; a refused packet carried no bytes, which is not the same as carrying zero.
4. UPPER-CASE THE ACTION ONCE, HERE. Exactly 2,700 lines write accept in lower case, and they are every accept ever sent by 10.30.0.5, 10.30.0.9 and 10.30.0.13. Compare case-sensitively and those three look like sources that were never allowed through: 540 extra denies land in question 3 and 27.5% becomes 32.0%.

GOLD – THE SHAPE THAT TRAVELS

`gold_source_minute`, one row per source IP per minute: `src_ip`, `minute_ts`, events, `distinct_dst_ports`, `distinct_dst_ips`, denies, accepts, `deny_ratio`, `is_internal` - exactly 36,295 rows. That is 200 chatter sources x 180 minutes, plus 180 for the backup server, plus 10, 90 and 15 for the three scanners, which only exist while they are scanning. Keep minutes with one event; `SUM(events)` over the table is 224,700.

THE TRAP MOST PEOPLE FALL INTO

Calling 10.30.8.20 a scanner. It clears 100 distinct ports in 177 five-minute windows, more than either real scanner, because a backup agent opens a fresh high port per stream. But 5,340 of its 5,400 connections were accepted, and a port is only evidence when the far side refused it. The rule that survives is denied with no accept ever, not port count. That threshold also misses 198.51.100.22: 900 ports over 90 minutes is 50 in any five-minute window, under any threshold you would pick, and it is a scanner.

THE THREE QUESTIONS YOUR GOLD LAYER MUST ANSWER

1. Distinct destination ports per source IP per minute, and the top offenders
2. Which sources hit more than 100 distinct ports in any 5-minute window?
3. What share of denied traffic comes from sources that never had a single accept?

IN SNOWFLAKE – THE HARD ONE, WORKED

Tables this query needs in Snowflake: **SILVER_FW**, **GOLD_SOURCE_MINUTE**

```
-- COUNT(DISTINCT ...) is not allowed as a window function in Snowflake, so a
-- sliding five-minute distinct-port count is two steps: reduce to one row per
-- source, minute and port, then re-count inside each window with a self-join.
WITH sp AS (SELECT DISTINCT src_ip, minute_ts, dst_port
  FROM SILVER_FW WHERE dst_port IS NOT NULL),
sm AS (SELECT DISTINCT src_ip, minute_ts FROM sp),
w AS (SELECT a.src_ip, a.minute_ts AS win_start,
  COUNT(DISTINCT b.dst_port) AS ports_5m
  FROM sm a JOIN sp b ON b.src_ip = a.src_ip
  AND b.minute_ts >= a.minute_ts
  AND b.minute_ts < DATEADD(minute, 5, a.minute_ts)
  GROUP BY 1, 2)
SELECT w.src_ip, MAX(w.ports_5m) AS peak_ports_5m,
  COUNT_IF(w.ports_5m > 100) AS windows_over_100,
  MAX(t.accepts) AS accepts_all_day
FROM w JOIN (SELECT src_ip, SUM(accepts) AS accepts
  FROM GOLD_SOURCE_MINUTE GROUP BY 1) t ON t.src_ip = w.src_ip
GROUP BY 1
HAVING MAX(w.ports_5m) > 100
ORDER BY peak_ports_5m DESC;
```

YOU ARE DONE WHEN

- ☐ Bronze 233,200 equals the file's line count; 230,700 after 2,500 unparsed; 224,700 after 6,000 duplicate eids. Print all four numbers and close the subtraction in code.
- ☐ Silver has exactly 204 distinct src_ip. DENY 12,000 plus ACCEPT 212,700 equals 224,700, and exactly 1,440 rows carry a NULL dst_port.
- ☐ gold_source_minute has exactly 36,295 rows and SUM(events) = 224,700. The peak distinct_dst_ports is 120 for 203.0.113.45, then 80, then 30; every chatter source peaks at 6, and the eight ICMP senders at 5.
- ☐ The query returns exactly 3 rows: 203.0.113.45 at 600 ports, 10.30.7.99 at 400, 10.30.8.20 at 150 - and only the last has a non-zero accepts_all_day, 5,340. Question 3 is 3,300 / 12,000 = 27.5%.

IF YOU FINISH EARLY

Add a per-source day table: distinct ports over the whole three hours, denies, accepts, first and last seen. The slow scanner that no five-minute threshold catches appears immediately at 900 distinct ports and zero accepts.

THE SITUATION

The capacity report says the busiest circuit in the region averages 4.6 Mbps on a 10 Mbps link, so nobody orders an upgrade. It does not. A router interface counter does not report bandwidth - it reports a total that only climbs, until it passes 4,294,967,296 and starts again at zero, and the difference between two polls becomes about minus four billion bytes. A pipeline that drops negative rows throws away 992 of the day's samples, and they are the fastest ones. Corrected, six links sit above 80% of rated for 2,875 minutes. Dropped, they report 2,645.

WHERE THE DATA COMES FROM

There is no dependable public file of raw 32-bit interface counters. Vendor MIBs do define ifInOctets as a Counter32, so the wrap is real and every network engineer has met it, but the public archives are RRD or MRTG exports that already converted counters to rates and repaired the wraps before writing - the defect this project is about is gone before you can see it. Do not quote columns or row counts for a telemetry dump you have not opened. The generator below is the primary path.

GENERATE IT – THE EXACT SPECIFICATION

Seed 34. Two CSVs with headers. if_polls: 40 interfaces, RTR-01..RTR-10 x ge-0/0/1..4, 288 polls of 300 s on 2025-05-06: 11,520 rows plus 2,000 dups = 13,520. Columns device_id, interface_id, poll_ts, in_octets, out_octets, if_speed_mbps, admin_status, sys_uptime_s, poll_id. Rated by index: 10, 10, 50, 100 Mbps. Peak = polls 121-216. ge-0/0/1 on RTR-01..RTR-06 runs 90% of rated in peak, 25% off; all others 40%/15%. Per-poll octets: rated x 1e6 x util / 8 x 300; out_octets by half. Counters are 32-bit: stored value = running total mod 4,294,967,296, so in_octets wraps exactly 992 times and out_octets 479; 573 of the 992 fall on the ten 100 Mbps ports, 284 on the ten 50, 135 on the twenty 10. Exactly 4 reboots: RTR-03 at poll 150, RTR-06 at 60, RTR-07 at 200, RTR-09 at 250; each resets that device's 4 counters to a tenth of an interval and sys_uptime_s to 30: 16 reset rows. RTR-08 is admin down for polls 96-119: 96 rows, status 'down', frozen counters, if_speed_mbps 0. if_master: 40 rows - device_id, interface_id, site, rated_mbps, role; the 4 RTR-05 rows pad device_id with a trailing space. Duplicates: polls 0-49 repeated for all 40 interfaces, 2,000 rows, each keeping its poll_id.

GENERATOR CODE – RUN THIS FIRST, EXACTLY AS WRITTEN

This assumes MY_ID and VOL are already set, as on the second page of this document. Do not change the seed or any of the counts — your checks later depend on them.

```
W = 4294967296
PT = "partition by device_id, interface_id"
UB = f"{PT} order by p rows between unbounded preceding and current row"
i = spark.range(40).selectExpr("cast(1 + id div 4 as int) dv",
    "cast(pmod(id, 4) as int) ix",
    "format_string('RTR-%02d', 1 + id div 4) device_id",
    "format_string('ge-0/0/%d', pmod(id, 4) + 1) interface_id",
    "format_string('SITE-%02d', 1 + id div 4) site",
    "element_at(array(10, 10, 50, 100), pmod(id, 4) + 1) rated_mbps",
    "element_at(array('access', 'access', 'edge', 'core'), pmod(id, 4) + 1) role")
i.selectExpr("if(dv = 5, concat(device_id, ' '), device_id) device_id", "interface_id",
    "site", "rated_mbps", "role").write.mode('overwrite').options(header=True,
    ignoreLeadingWhiteSpace=False, ignoreTrailingWhiteSpace=False).csv(f'{VOL}/raw/if_master')
p = i.crossJoin(spark.range(288).withColumnRenamed('id', 'p'))
p = p.selectExpr("*(dv = 8 and p between 96 and 119) dn",
    "case dv when 3 then 150 when 6 then 60 when 7 then 200 when 9 then 250 end pr",
    "cast(rated_mbps * 1000000 * if(ix = 0 and dv <= 6,
    " if(p between 121 and 216, 90, 25), if(p between 121 and 216, 40, 15))"
    " / 800 * 300 as bigint) d0")
p = p.selectExpr("*(sum(if(dn, 0, d0)) over ({UB}) ci",
    "sum(if(dn, 0, d0) div 2) over ({UB}) co")
p = p.selectExpr("*(max(if(p = pr, ci, null)) over ({PT}) ri",
    "max(if(p = pr, co, null)) over ({PT}) ro", "max(if(p = pr, d0, null)) over ({PT}) rd")
p = p.selectExpr("device_id", "interface_id",
    "timestamp'2025-05-06 00:00:00' + make_dt_interval(0, 0, cast(5 * p as int)) poll_ts",
    "pmod(if(pr is not null and p >= pr, ci - ri + rd div 10, ci), {W}) in_octets",
    "pmod(if(pr is not null and p >= pr, co - ro + (rd div 2) div 10, co), {W})"
    " out_octets",
    "if(dn, 0, rated_mbps) if_speed_mbps", "if(dn, 'down', 'up') admin_status",
    "if(pr is not null and p >= pr, 30 + 300 * (p - pr), 100000 + dv * 1000 + 300 * p)"
    " sys_uptime_s", "format_string('PL%02d%d%03d', dv, ix, p) poll_id", "p")
(p.drop('p').union(p.filter('p < 50').drop('p')).write.mode('overwrite')
    .options(header=True, ignoreLeadingWhiteSpace=False, ignoreTrailingWhiteSpace=False)
    .csv(f'{VOL}/raw/if_polls'))
```

BRONZE – LAND IT, CHANGE NOTHING

Read both folders with header=True and every column as STRING, then stop. in_octets runs to 4,294,967,295, which does not fit an INT, so cast it in Silver with try_cast to BIGINT rather than letting an inferred schema decide. Bronze polls is exactly 13,520 rows and bronze master exactly 40. Do not join the two, do not dedupe, do not sort, and do not trim - the trailing space in four master rows is data you have to be able to see later.

SILVER – THE DECISIONS YOU MUST MAKE, AND RECORD

Each decision below has a number attached to it. If your number does not match, stop and find out why before you go on — that gap is the finding, not a nuisance.

1. DEDUPE ON poll_id BEFORE ANY LAG. 13,520 rows fall to exactly 11,520, so 2,000 go, and 40 interfaces x 288 polls = 11,520 closes it. Order matters here: leave the duplicates in and LAG compares a row with its own copy at the same poll_ts, the delta comes out 0, and polls 0-49 report about half the real throughput on every interface.
2. A NEGATIVE DELTA IS A WRAP, NOT A ROW TO DROP. Exactly 992 in_octets deltas and 479 out_octets deltas come out negative because the counter passed 4,294,967,296; add 4,294,967,296 to each and keep the row. The check that the fix is right: no corrected delta exceeds 1,500,000,000, which is the largest one interval can hold here.
3. A REBOOT IS NOT A WRAP. Exactly 16 rows - 4 devices x 4 interfaces - have a sys_uptime_s lower than the previous poll's. There the counter restarted near zero, so the interval is genuinely unknown: set the delta NULL and keep the row. Add 2^{32} there instead and you invent up to 4.29 GB in five minutes, a false 114 Mbps.
4. TRIM THE JOIN KEY AND DISTRUST if_speed_mbps. Exactly 4 master rows, RTR-05's, carry a trailing space in device_id, so an untrimmed join silently drops 4 interfaces - one of them RTR-05 ge-0/0/1, which is one of the six links question 2 is about, so the answer comes back 5. And 96 polls report if_speed_mbps 0 while RTR-08 is down; take rated_mbps from the master and wrap it in NULLIF.

GOLD – THE SHAPE THAT TRAVELS

gold_iface_interval, one row per device_id, interface_id and 5-minute interval, keyed on the interval's starting poll: device_id, interface_id, site, role, rated_mbps, bucket_start, in_octets_delta, out_octets_delta, in_mbps, out_mbps, util_pct, wrap_flag, reboot_flag - exactly 11,480 rows, 40 interfaces x 287 intervals. The first poll of each interface has no predecessor and makes no row. 16 rows carry reboot_flag true with NULL deltas; 992 carry wrap_flag true.

THE TRAP MOST PEOPLE FALL INTO

WHERE delta > 0. It reads like hygiene and it is a 992-row hole in exactly the wrong places, because the busiest intervals are the ones that wrap. Each of the six 90%-loaded links then reports 440 minutes above 80% instead of 480, and its daily mean falls 2.2%; the 100 Mbps links lose 6.8%. The other half is correcting every negative: 16 of the 1,008 are reboots, where the counter restarted rather than rolled over, and adding 2^{32} there books 4.29 GB in five minutes - a false 114 Mbps on a 10 Mbps circuit.

THE THREE QUESTIONS YOUR GOLD LAYER MUST ANSWER

1. Throughput in Mbps per interface per 5-minute bucket
2. Which interfaces run above 80% of their rated speed, and for how many minutes a day?
3. Which interfaces show the most counter resets, and are those the busiest ones?

IN SNOWFLAKE – THE HARD ONE, WORKED

Tables this query needs in Snowflake: **SILVER_POLL**, **DIM_INTERFACE**

```
-- LAG gives the raw delta. A negative one is a wrap, UNLESS sys_uptime_s fell:
-- then the device rebooted and the interval is unknown, not 2^32 larger.
WITH d AS (SELECT device_id, interface_id, poll_ts,
    in_octets - LAG(in_octets) OVER (PARTITION BY device_id, interface_id
    ORDER BY poll_ts) AS raw_d,
    sys_uptime_s - LAG(sys_uptime_s) OVER (PARTITION BY device_id, interface_id
    ORDER BY poll_ts) AS up_d
FROM SILVER_POLL),
c AS (SELECT d.*, CASE WHEN up_d < 0 THEN NULL
    WHEN raw_d < 0 THEN raw_d + 4294967296 ELSE raw_d END AS octets FROM d)
SELECT c.device_id, c.interface_id, m.rated_mbps,
    COUNT_IF(c.raw_d < 0 AND c.up_d >= 0) AS wraps,
    COUNT_IF(c.up_d < 0) AS reboots,
    ROUND(MAX(c.octets * 8.0 / 300 / 1e6 / NULLIF(m.rated_mbps, 0)) * 100, 1) AS peak_pct
FROM c JOIN DIM_INTERFACE m
    ON TRIM(m.device_id) = c.device_id AND TRIM(m.interface_id) = c.interface_id
GROUP BY 1, 2, 3
ORDER BY wraps DESC, peak_pct DESC;
```

YOU ARE DONE WHEN

- ☐ Bronze polls 13,520, Silver 11,520 after exactly 2,000 duplicate poll_ids, and 40 x 288 = 11,520 closes it. The master has 40 rows and 40 distinct trimmed device_id plus interface_id pairs.
- ☐ Negative in_octets deltas number exactly 1,008: 992 wraps plus 16 reboots. After the correction no delta is negative, none exceeds 1,500,000,000, and none implies more than 100% of rated speed.
- ☐ gold_iface_interval has exactly 11,480 rows, 16 of them with a NULL in_mbps. Intervals above 80% of rated total exactly 575 - 96 on each of five links and 95 on RTR-03 ge-0/0/1, which loses one to its reboot - which is 2,875 minutes.
- ☐ The query returns 40 rows. The ten 100 Mbps core interfaces lead on wraps with 55 to 58 each; the six links running at 90% of rated show only 10 or 11. Wraps count bytes, not utilisation - that is the answer to question 3, and it is counter-intuitive.

IF YOU FINISH EARLY

Add one data-quality column: the implied Mbps of every corrected delta against rated_mbps, flagged above 100%. A correct wrap fix never produces one, so an empty flag list is the cheapest proof the arithmetic held.